

Verification Continuum™

Verdi® Fault Analysis and Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	5
Synopsys Statement on Inclusivity and Diversity	5

1. Fault Analysis and Debug	7
Overview	7
Analyzing Fault Coverage	7
Enabling the Fault Coverage View	8
Loading a Fault Coverage Database in the GUI	9
Overview of Fault Coverage Windows	10
Configuring Status Color and Weight	12
Configuring Status Columns	13
Viewing Faults by Type	16
Viewing Faults by Class	16
Using the FaultSummary Window	16
Checking the Coverage of a Design Hierarchy	17
Viewing the Coverage of a Module	17
Searching Within a Design and Its Modules	18
Applying Tests on a Design and Its Modules	19
Using the FaultDetail Window	20
Grouping Faults	21
Managing Exclusions in Fault Coverage	22
Configuring the Color of an Exclusion Type	23
Excluding Faults	24
GUI Updates	24
FaultSummary Window	25
FaultDetail Window	26
Support for GM+mFM FSDB File in Verdi	26
Selecting Fault ID From nTrace Window	26
Displaying Fault ID of Secondary FSDB File in nTrace Window Title	30
Selecting Fault ID From nWave Window	31
Opening Same FSDB File in Different nWave Window	33
Save Restore Session	33
Changing Fault ID	34
Reopening FSDB File in nWave	34

Contents

Adding Fault Waveform	35
Limitations	36
nWave Display for GM/FM Waveform	38
Display of Hold Time Pair for the Fault	39
Comparing FSDB files Containing GM+nFM Data in nCompare	41

Preface

Verdi provides fault analysis and debug support through the *nTrace* and *nWave* windows. This document provides detailed information about the fault analysis and debug features supported by Verdi.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to support_center@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to support_center@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary

Contents

language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Fault Analysis and Debug

Verdi provides fault analysis and debug support through the *nTrace* and *nWave* windows.

Overview

This chapter contains the following sections:

- [Analyzing Fault Coverage](#)
 - [Managing Exclusions in Fault Coverage](#)
 - [Support for GM+mFM FSDB File in Verdi](#)
 - [nWave Display for GM/FM Waveform](#)
-

Analyzing Fault Coverage

Verdi provides an interface to explore the quality of fault coverage of a design. Using this interface, you can perform the following tasks:

- Examine fault coverage, test coverage, undetected faults, and the total number of faults.
- View faults based on their types, such as Stuck at Zero, Stuck at One, Toggle, and so on.
- Infer faults based on their classes, such as Port, Wire, Array, and so on.
- Quickly scan a design for areas of low coverage using coverage bars.
- Adjust the weights of individual statuses in coverage calculations.
- Check coverage details in the *Fault* and *Unselected* panes.

For more details, see the following sections:

- [Enabling the Fault Coverage View](#)
- [Loading a Fault Coverage Database in the GUI](#)
- [Overview of Fault Coverage Windows](#)

- [Using the FaultSummary Window](#)
- [Using the FaultDetail Window](#)

Note:

Verdi calls the Fast Signal Database (FSDB) dumping script directly from `$VCS_HOME` if the `VERDI_FUSA_DUMP_FSDB_CMD` environment variable is not set and if the fault simulation results are written to the fault database by VC Z01X.

The script is located at:

```
$VCS_HOME/etc/fcm/dump_fm_fsdb_from_verdi.csh
```

Enabling the Fault Coverage View

To enable the fault coverage view in Verdi, specify the fault database, project name and campaign name by either of the following:

1. Environment variables:

```
setenv SNPS_FDB_SERVER <fault_database_servername>
setenv SNPS_FDB_PROJECT <project_name>
setenv SNPS_FDB_FAULT_CAMPAIGN <campaign_name>
setenv SNPS_FDB_STORAGE_PATH <storage_path>
```

2. Command-line options:

```
-fdb_path <fault_database_directory_path>
-fdb_project <fault_database_project_name>
-campaign <campaign_name>
```

In releases earlier than U-2023.03, the following command-line options are supported:

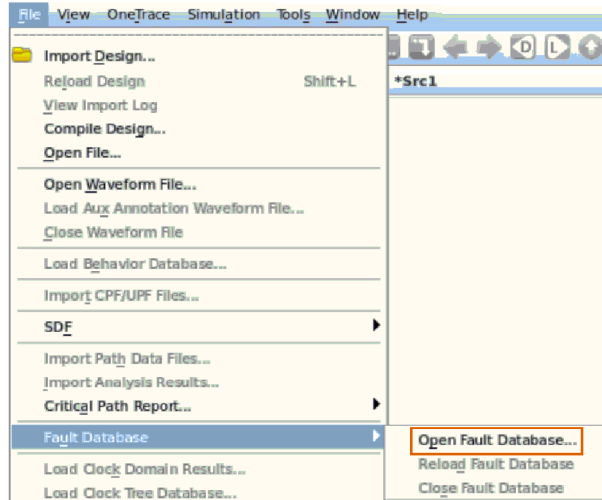
```
-fdb_server <fault_database_server_name>
-fdb_name <project_name>
-campaign <campaign_name>
```

If the FDB server is not specified, Verdi loads the FDB in local mode instead of server mode.

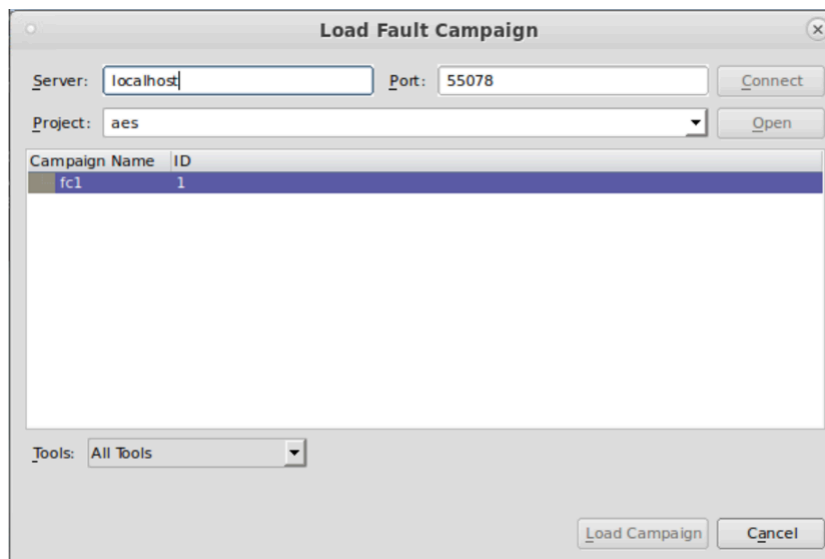
Loading a Fault Coverage Database in the GUI

You can load a fault coverage database in the Verdi GUI if you do not specify the database at the command line while invoking Verdi. To load a fault coverage database in the Verdi GUI, perform the following steps:

1. Load a fault coverage database in Verdi by clicking **File > Fault Database > Open Fault Database** as shown in the following figure:



2. The **Load Fault Campaign** dialog box is displayed to specify server name, port name, project, and campaign name. The following figure shows the **Load Fault Campaign** dialog box:



Note:

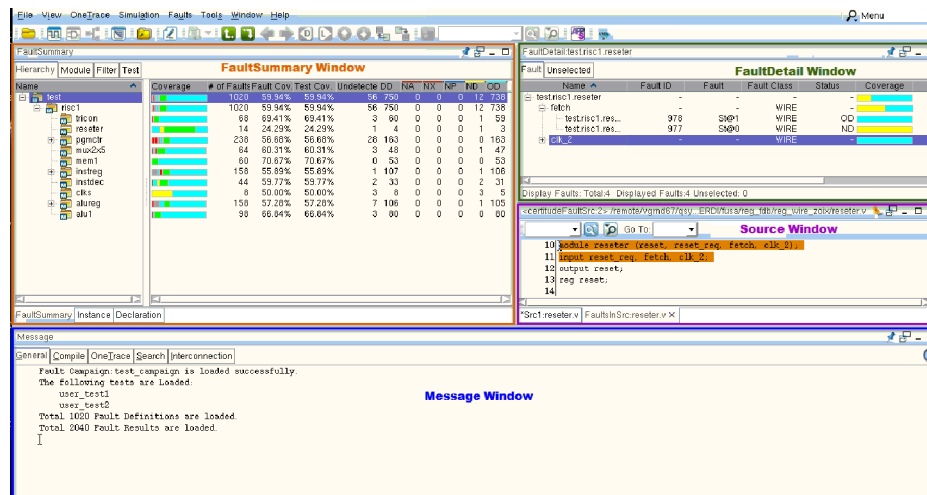
You can use **Faults > Reload Database** and **Faults > Close Database** to reload and close a database respectively. If you close a database, the **Source** pane is removed from the GUI. You can display this pane again using the **Windows Manager** dialog box.

Overview of Fault Coverage Windows

When the Fault Coverage view is enabled, Verdi displays the following two new windows to view fault coverage details:

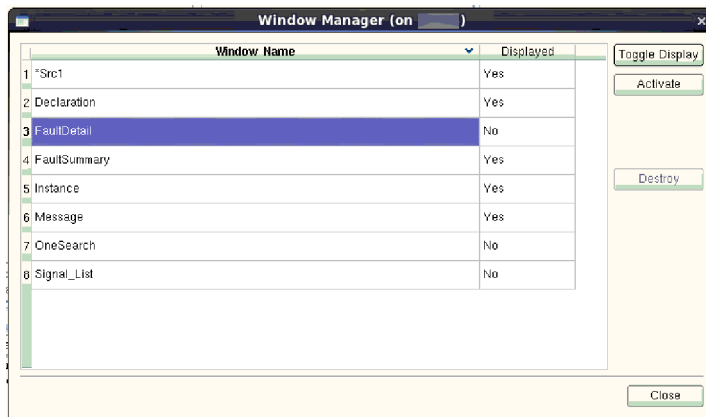
- FaultSummary Window
- FaultDetail Window

The following figure shows these windows along with Source and Message windows:

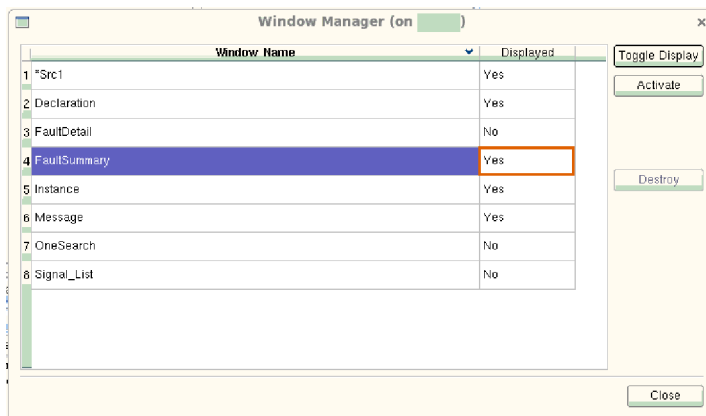


If these windows are not displayed, then you can display these windows using the following steps:

1. Click **Window > Window Manager** to display the **Window Manager** dialog box as follows:



2. Select a window name to display or hide it and click **Toggle Display**. For example, to display the **FaultDetail** window, select it and click **Toggle Display**. If a window's displayed value becomes Yes, it gets displayed as highlighted in the following figure:



Note:

To bring focus on any window, select a window from the **Window Manager** and click **Activate**. On the other hand, to remove any window from the GUI, select a window from the **Window Manager** and click **Destroy**.

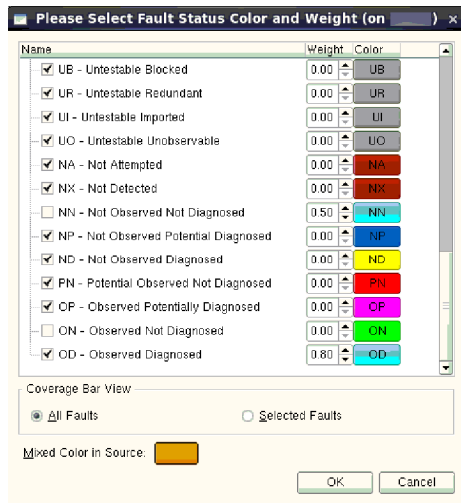
The Fault Coverage view enables you to configure the color, weight, and columns of fault statuses. In addition, it enables you to view faults based on their types and classes. For more details, see the following sections:

- [Configuring Status Color and Weight](#)
- [Configuring Status Columns](#)
- [Viewing Faults by Type](#)
- [Viewing Faults by Class](#)

Configuring Status Color and Weight

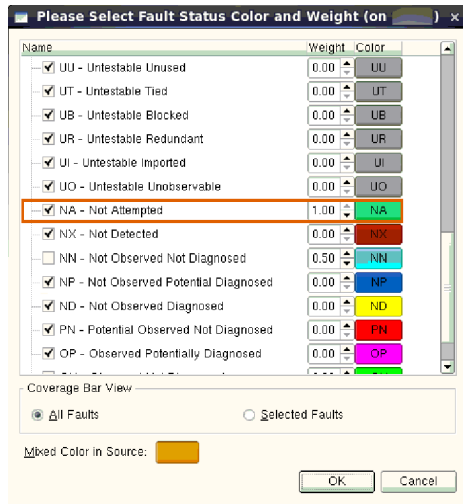
To configure the color and weight of fault types, perform the following steps:

1. Click **Faults > Configure Status Color/Weight** to display the following dialog box:



2. To update the weight of any fault type, specify the updated value in the **Weight** field or use the up and down arrows to adjust the weight. On the other hand, to update the color of any fault type, click a color to bring up the **Select Color** dialog box. Select a standard color, choose from a custom color, or use the color spectrum tool to define a

new color. For example, update the value of weight and color of **NA-Not Attempted** to 1.00 and green respectively as shown in the following figure:



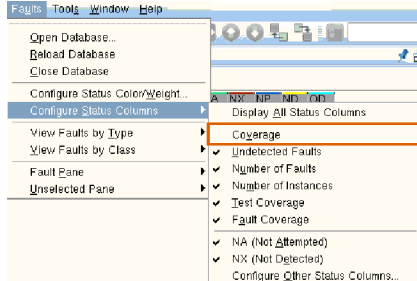
3. Apply color and weight updates for all faults or only selected faults using either **All Faults** or **Selected Faults** respectively.
4. Click **OK**.

Configuring Status Columns

Fault status columns, such as **Coverage**, **# of Faults**, **Fault Coverage**, **Test Coverage**, and **Undetected** are displayed, by default. In addition, fault types, such as **DD** (Dropped Detected), **NA** (Not Attempted), **NX** (Not Detected), **NP** (Not Observed Potential Diagnosed), **ND** (Not Observed Diagnosed), and **OD** (Observed Diagnosed) are displayed, by default as shown in the following figure:

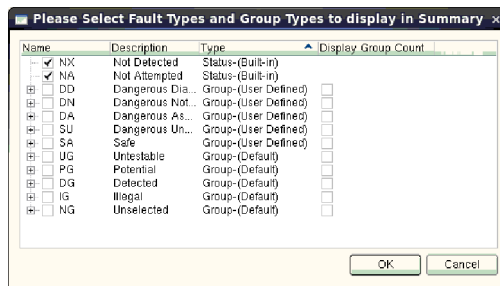
Coverage	# of Faults	Fault Cov.	Test Cov.	Undetected	DD	NA	NX	NP	ND	OD
	1020	59.94%	59.94%	56	750	0	0	0	12	738
	1020	59.94%	59.94%	56	750	0	0	0	12	738
	68	69.41%	69.41%	3	60	0	0	0	1	59
	14	24.29%	24.29%	1	4	0	0	0	1	3
	238	56.66%	56.66%	28	163	0	0	0	0	163
	64	60.31%	60.31%	3	48	0	0	0	1	47
	60	70.67%	70.67%	0	53	0	0	0	0	53
	158	55.69%	55.69%	1	107	0	0	0	1	106
	44	59.77%	59.77%	2	33	0	0	0	2	31
	8	50.00%	50.00%	3	6	0	0	0	3	5
	158	57.26%	57.26%	7	106	0	0	0	1	105
	98	66.64%	66.64%	3	80	0	0	0	0	80

You can hide any default status column by unselecting it from **Faults > Configure Status Columns**. For example, to hide the **Coverage** column, unselect **Coverage** from **Faults > Configure Status Columns** as shown in the following figure:



To display all status columns, select **Faults > Configure Status Columns** and then select **Display All Status Columns**. To configure remaining status columns, perform the following steps:

1. Select **Faults > Configure Other Status Columns** to display the following dialog box:



2. Select the fault status column that you want to display and click **OK**. Note that **NX** and **NA** are selected, by default. You can unselect them as per your requirement.

The following table explains default status columns:

Column Name	Description
Coverage	This column shows coverage bars. These bars provide a visualization of the distribution of fault statuses in an instance or a module. Each section of the bar displays a color associated with a fault status or group of fault statuses. The size of each colored section is relative to the total number of faults in the instance or module. For example, an instance with 300 faults, 100 of which are ND, and an instance with 3 faults, 1 of which is ND, both display a coverage bar that is one-third red for ND. Parent instances are cumulative of their child instances. For example, the Coverage Bar for a parent instance might display one-third red for ND and two thirds green for DD. Expanding the parent instance might display three instances, one with a red Coverage Bar and two with green bars.

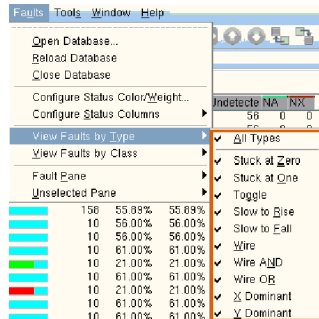
Column Name	Description
# of Faults	This column shows the total number of faults in a module or an instance.
Test Cov.	This column shows fault detect percentage of all faults excluding untestable faults. Untestable faults are Untestable Unused (UU), Untestable Tied (UT), Untestable Blocked (UB), Untestable Redundant (UR), Untestable Imported (UI), and Untestable Unobservable (UO). Test coverage is calculated as follows: $\text{Test coverage} = \frac{\text{Weighted_Total}}{\text{Total}}$ where, Weighted_Total is the total of each fault status multiplied by its weight value: $\text{Weighted_Total} = (\text{DD} * \text{DD_weight}) + (\text{DT} * \text{DT_weight}) + (\text{PD} * \text{PD_weight}) + (\text{PT} * \text{PT_weight}) + (\text{NA} * \text{NA_weight}) + \dots$ Default weights: 1.0 - DD, DT 0.5 - PD, PT 0.0 - NA, ND, DO, PD, PT, PO, OZ, OA, HA, HM, HT, --, IA, IX, NC, NO, NS, NI, NT, UU, UT, UB, UR, UI, UO, EN
Fault Cov.	This column shows fault detect percentage of all faults including untestable faults. Fault coverage is calculated as follows: $\text{Fault coverage} = \frac{\text{Weighted_Total}}{(\text{Total} + \text{UB} + \text{UT} + \text{UU} + \text{UR} + \text{UO} + \text{UI})}$ where, Weighted_Total is the total of each fault status multiplied by its weight value: $\text{Weighted_Total} = (\text{DD} * \text{DD_weight}) + (\text{DT} * \text{DT_weight}) + (\text{PD} * \text{PD_weight}) + (\text{PT} * \text{PT_weight}) + (\text{NA} * \text{NA_weight}) + \dots$ Default weights: 1.0 - DD, DT 0.5 - PD, PT 0.0 - NA, ND, DO, PD, PT, PO, OZ, OA, HA, HM, HT, --, IA, IX, NC, NO, NS, NI, NT, UU, UT, UB, UR, UI, UO, EN
Undetected Faults	This column displays the number of undetected faults in each instance or a module. Undetected faults are the faults that are not detected. These faults are as follows: • Oscillating faults [Oscillating Active (OA), Oscillating Zero (OZ)] • Hyperfaults [Hyperactive (HA), Hypermemory (HM), Hypertrophic (HT)] • Other testable faults [Illegal Access (IA), Illegal User PLI (IP), Illegal Hierarchical Reference (IH), Illegal File I/O (IF), Impossible X-State (IX), Not Observed Diagnosed (ND), Not Attempted (NA), Not Controlled (NC), Not Observed (NO), Not Strobed (NS), Not Injected (NI), Not Tested (NT)]

Note:

To know details about any fault type, see the *Z01X Simulator Safety Verification User Guide*.

Viewing Faults by Type

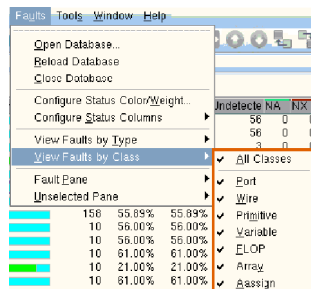
To view faults based on their types, click **Faults > Viewing Faults by Type** to display the following fault types:



To know details about fault types, see the *Z01X Simulator Safety Verification User Guide*.

Viewing Faults by Class

To view faults based on their fault classes, click **Faults > Viewing Faults by Class** to display the following fault classes:



To know details about fault classes, see the *Z01X Simulator Safety Verification User Guide*.

Using the FaultSummary Window

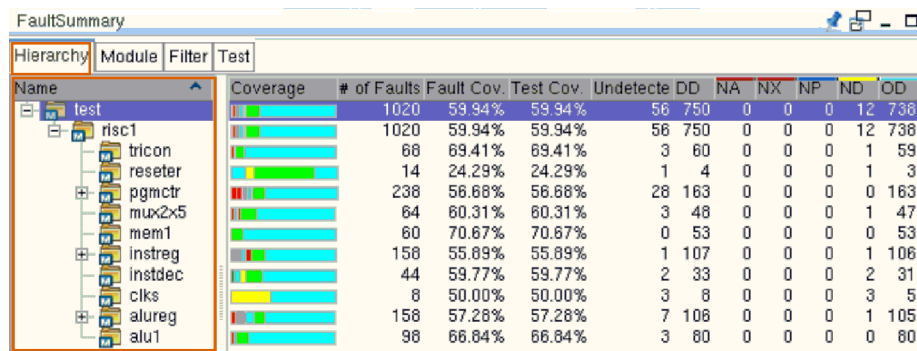
The **FaultSummary** window displays fault and source information for modules, their instances, and signals. Click a module, its instance, or a signal in the **FaultSummary** window to make it the focus in the **FaultDetail** and **Source** windows.

The **FaultSummary** window consists of four panes: **Hierarchy**, **Module**, **Filter**, and **Test**. For details on these four panes, see the following sections:

- [Checking the Coverage of a Design Hierarchy](#)
- [Viewing the Coverage of a Module](#)
- [Searching Within a Design and Its Modules](#)
- [Applying Tests on a Design and Its Modules](#)

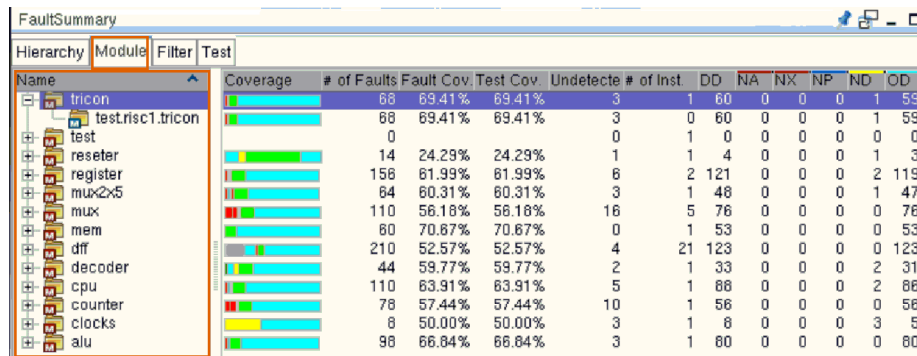
Checking the Coverage of a Design Hierarchy

To check the coverage of a design hierarchy, see the **Hierarchy** pane in the **FaultSummary** window. For example, the following figure shows a design hierarchy of test in the **Hierarchy** pane:



Viewing the Coverage of a Module

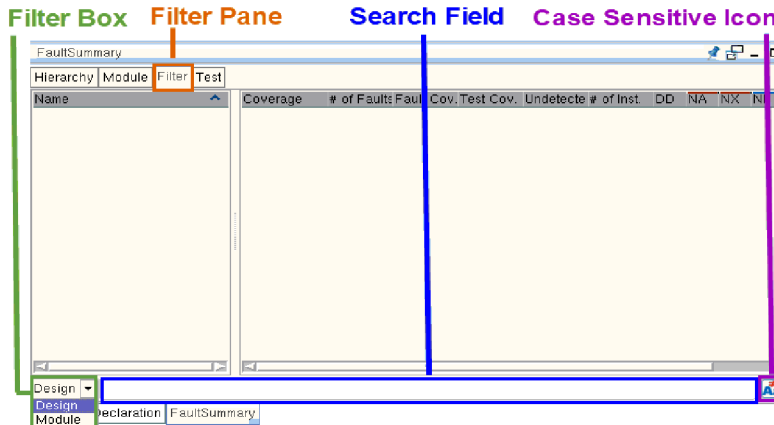
To view the coverage of a module, see the **Module** pane in the **FaultSummary** window. The **Module** pane shows the definition and instance of all modules in a design. For example, the following figure shows the definition and instance of modules, such as tricon, test, reseter, register, and so on:



Searching Within a Design and Its Modules

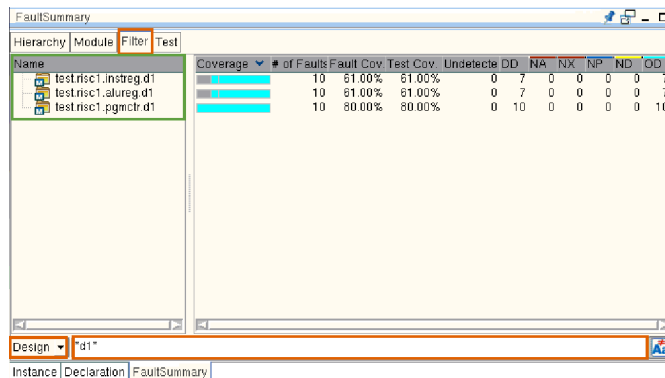
To search within a design or only in modules in the design, perform the following steps:

1. Select the **Filter** pane in the **FaultSummary** window.
2. Specify whether a search needs to be performed in a design or a module. For this, select **Design** or **Module** respectively from the **Filter** box as highlighted in the following figure:

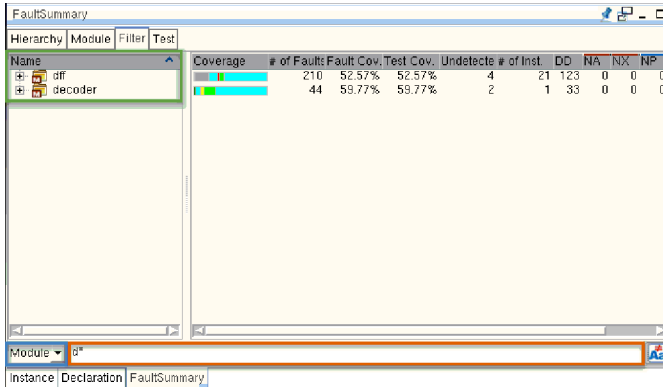


In the **Filter** pane, you can use any wild card to specify any search criteria in the **Search** field. You can make the **Search** field case-sensitive using the **Case Sensitive** icon as highlighted in the figure above.

For example, if you specify ***d1*** in the **Search** field and select **Design**, Verdi displays its search results from the design hierarchy as highlighted in green in the following figure:



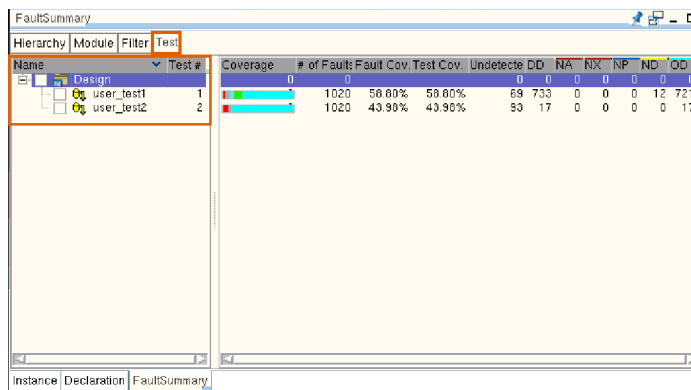
Similarly, you can search all modules that start with `d` in a design by specifying its search criteria in the **Search** field and selecting **Module** from the **Filter** box as highlighted in the following figure:



Applying Tests on a Design and Its Modules

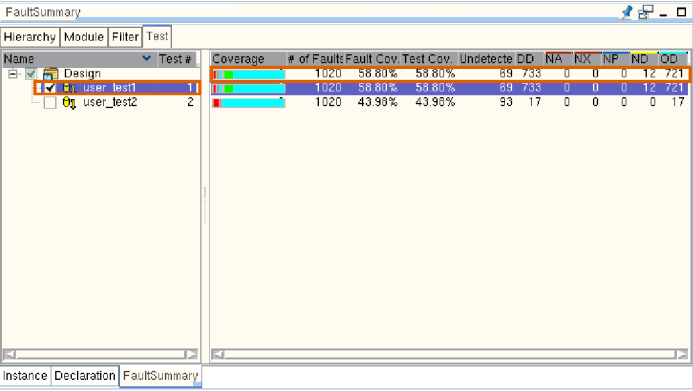
When a design is loaded in Verdi, its tests automatically get loaded. You can select a test to view its impact on coverage. To apply a test on a design and its modules, perform the following steps:

1. Select the **Test** pane in the **FaultSummary** window to show **Name** and **Test #** columns as shown in the following figure:

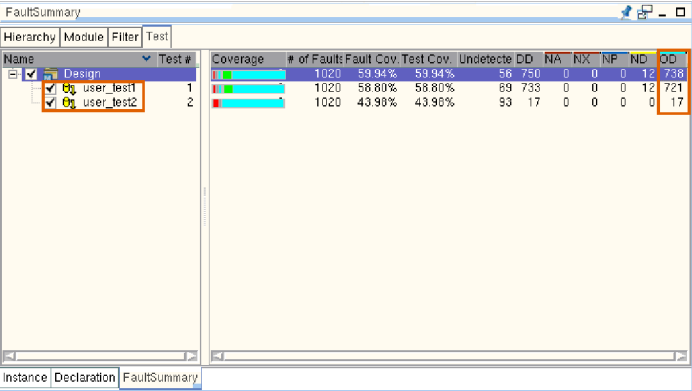


2. Select a test that you want to apply on a design and its modules. For example, to apply `user_test1` on the design, select the check box corresponding to `user_test1`. Note

that the value of columns, such as **Coverage**, **# of Faults**, **Undetected**, and so on is 0 because no test is applied. If you select `user_test1`, the value of these columns is updated as highlighted in the following figure:



If multiple tests are applied on a design, the value of coverage and fault types in a design is calculated as the sum of their values for each test. For example, consider that both `user_test1` and `user_test2` tests are selected. If **OD** in `user_test1` is 721 and **OD** in `user_test2` is 17, the value of **OD** in the design becomes 738 as shown in the following figure:



Using the FaultDetail Window

When an instance or a module is selected in the **FaultSummary** window, its details are populated in the **FaultSummary** window. In its title bar, the **FaultSummary** window displays the name of the instance or module that is loaded.

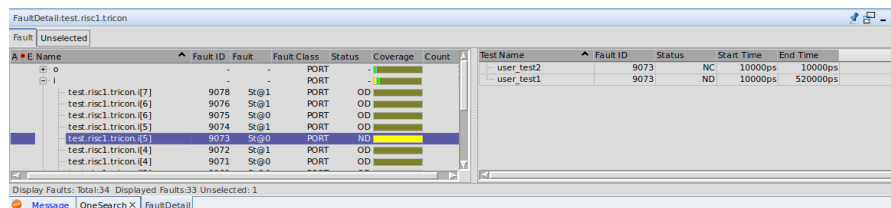
The **FaultDetail** window consists of two panes: **Fault Pane** and **Unselected Pane**.

All selected faults will be in the **Fault Pane** and unselected faults will be in the **Unselected Pane**.

Fault details of the fault statuses shown in the **FaultSummary** window are displayed in the **Fault** pane. For each fault, the following fault details will be shown:

- For faults except transient:
 - Fault details include fault ID, fault name, fault type, fault class, fault status, coverage bar, fault count, start time, and end time.
- For transient faults:
 - Fault details include fault type, fault class, fault location, coverage bar, fault count, cycle number, clock name, fault injection time, and origin level.

The following figure shows the fault details of the `PORT` faults in `test.risc1.tricon`. Once a fault is selected, the tests and the corresponding fault status will be shown on the right side.



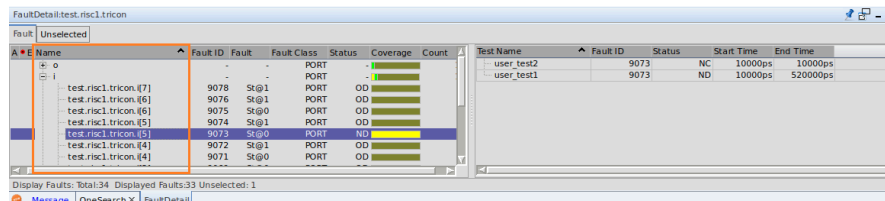
Grouping Faults

In the **Fault** pane, you can group faults by signals, location, and instance. By default, faults are grouped by fault location.

To group faults in the **Fault** pane, select any of the following options from **Faults > Fault Pane**:

• Group Faults by Location

This option groups faults by location. For example, if you see the detail of `test.risc1.tricon` in the **Fault** pane, the **Name** column displays the locations of faults as highlighted in the following figure:



• Group Faults by Instance

This option groups faults by instances. For example, if you see the detail of `test.risc1.tricon` in the **Fault** pane, the **Name** column groups faults based on instances as highlighted in the following figure:

Name	Fault ID	Fault	Fault Class	Status	Coverage	Count	Start Time	End Time
test.risc1.tricon	-	-	-	-	-	31		
o	-	-	PORT	-	-	15		
i	-	-	PORT	-	-	15		
ctl	-	-	PORT	-	-	1		

Display Faults: Total:32 Displayed Faults:31 Unselected: 1

- **Group Faults by Signal**

This option groups faults by signals. For example, if you see the detail of `test.risc1.tricon` in the **Fault** pane, the **Name** column groups faults based on signals as highlighted in the following figure:

Name	Fault ID	Fault	Fault Class	Status	Coverage	Count	Start Time	End Time
tricon.o	-	-	-	-	-	15		
test.risc1.tricon.o	-	-	PORT	-	-	15		
tricon.i	-	-	-	-	-	15		
test.risc1.tricon.i	-	-	PORT	-	-	15		
tricon.ctl	-	-	-	-	-	1		
test.risc1.tricon...	-	-	PORT	-	-	1		

Display Faults: Total:32 Displayed Faults:31 Unselected: 1

Managing Exclusions in Fault Coverage

When you identify a fault that can be ignored, or is impossible by design, you can mark it for exclusion directly in the GUI and add a comment explaining why it is excluded. You can then save those exclusions and comments to an exclude file that can be reused in the next and future sessions. Excluded objects are removed from the coverage score calculation.

Verdi provides exclusions support in the Fault Coverage view, by default. To enable the fault coverage view in Verdi, see [Enabling the Fault Coverage View](#).

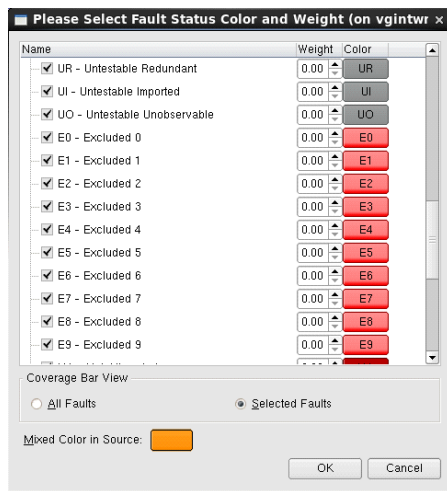
Verdi enables you to set the color of each exclusion type. If you load any exclusion or create any new exclusion, then few exclusion columns are displayed in the **FaultSummary** and **FaultDetail** Windows. For more details, see the following sections:

- [Configuring the Color of an Exclusion Type](#)
- [Excluding Faults](#)
- [GUI Updates](#)

Configuring the Color of an Exclusion Type

To configure the color of an exclusion type, perform the following steps:

1. Click **Faults > Configure Status Color/Weight** to display the following dialog box:

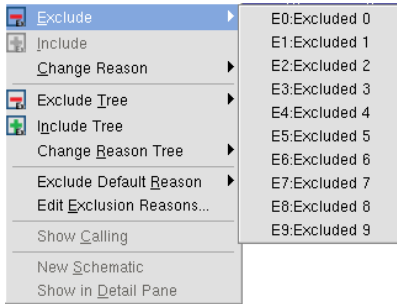


In the example, E0 to E9 indicates exclusion types (Excluded 0 to Excluded 9). Note that the weight setting is disabled for exclusion types.

2. To change the color of any exclusion type, click a color to bring up the **Select Color** dialog box. Select a standard color, choose from a custom color, or use the color spectrum tool to define a new color.
3. Apply color updates for all faults or only selected faults using either **All Faults** or **Selected Faults** respectively.
4. Click **OK**.

Excluding Faults

To exclude a fault, right-click the fault and click **Exclude** to choose the exclusion type to be excluded, as shown here:



The following table explains the menu commands added for exclusions support:

Table 1 *Menu Commands for Exclusions Support*

Command	Description
Exclude	Enables you to select the exclusion type for faults.
Include	Enables you to include back faults for coverage calculation. This command is enabled only when an excluded or excluding object is selected.
Change Reason	Enables you to change the exclusion type. This command is enabled only when an excluded or excluding object is selected.
Exclude Default Reason	Enables you to set an exclusion type as default.
Edit Exclusion Reason	Enables you to set description for each excluded type (Excluded 0 to Excluded 9).

GUI Updates

Following windows are updated to provide exclusions support:

- [FaultSummary Window](#)
- [FaultDetail Window](#)

FaultSummary Window

In the **FaultSummary** Window, a column is added for each selected excluded type (E0 to E9) and one column **EG** is added to view the total number of exclusions. The following figure shows the **FaultSummary** Window:

Name	Coverage	# of Faults Undetecte	EG	E0	E8	E9	NA	NX
test	701	23	15	13	0	2	0	0
risc1	701	23	15	13	0	2	0	0
tricon	34	1	0	0	0	0	0	0
reseter	10	0	0	0	0	0	0	0
pgmctr	101	14	15	13	0	2	0	0
m5	7	4	1	1	0	0	0	0
m4	8	1	0	0	0	0	0	0
m3	0	0	8	8	0	0	0	0
m2	8	1	0	0	0	0	0	0
m1	6	1	2	0	0	2	0	0
d5	10	2	0	0	0	0	0	0
d4	10	0	0	0	0	0	0	0
d3	8	0	2	2	0	0	0	0
d2	10	0	0	0	0	0	0	0
d1	10	0	0	0	0	0	0	0
mux2x5	32	1	0	0	0	0	0	0
mem1	94	0	0	0	0	0	0	0

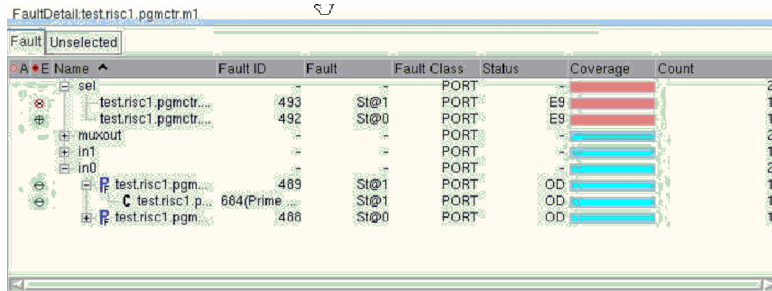
Note that two columns **A Attempted** and **E Excluded** are also included in the **FaultSummary** window to show the warning mark and the exclusion mark corresponding to each fault respectively. If you do not want any fault to be excluded unexpectedly, you can tell Verdi to display a warning if any such fault is marked for exclusion. For displaying the warning, select the **Warning If Excluded** column corresponding to a fault, as shown in the following dialog box:

Name	Description	Type	Display Group Count	Warning If Excluded
☒ NA	Not Attempted	Status-(Built-in)	☐	☐
☒ NX	Not Detected	Status-(Built-in)	☐	☐
☒ --	Collapsed	Status-(Built-in)	☐	☐
☐ ET	Excluded Fault	Group-(User Defined)	☐	☐
☐ DD	Dangerous Diagnosed	Group-(User Defined)	☐	☐
☐ OD	Observed Diagnosed	Status-(User Defined)	☐	☒
☐ NP	Not Observed Potential ...	Status-(User Defined)	☐	☒
☐ ND	Not Observed Diagnosed	Status-(User Defined)	☐	☐
☐ SU	Dangerous Unobserved	Group-(User Defined)	☐	☐
☐ NN	Not Observed Not Diag...	Status-(User Defined)	☐	☐
☒ NT	Not Tested	Status-(Built-in)	☐	☐
☒ NO	Not Observed	Status-(Built-in)	☐	☐
☒ NC	Not Controlled	Status-(Built-in)	☐	☐
☒ SA	Safe	Group-(User Defined)	☐	☐
☒ DA	Dangerous Assumed	Group-(User Defined)	☐	☐

For more details on the **FaultSummary** Window, see [Using the FaultSummary Window](#).

FaultDetail Window

In the **FaultDetail** Window, two columns  **Attempted** and  **Excluded** are included. The following figure shows the **FaultDetail** Window:



The screenshot shows the 'FaultDetail' window for a file named 'test.risc1.pgmctr.m1'. It displays a table with columns: Name, Fault ID, Fault, Fault Class, Status, Coverage, and Count. The table lists several faults, including 'test.risc1.pgmctr...' with IDs 493 and 492, 'muxout', 'in1', 'in0', and 'test.risc1.pgm...' with IDs 489, 664(Prime), and 488. The 'Status' column shows 'PORT' for most faults, and 'Coverage' shows 'E9' or 'OD'. The 'Count' column shows values like 2, 1, 2, 1, 2, 1, 1, 1.

Name	Fault ID	Fault	Fault Class	Status	Coverage	Count
test.risc1.pgmctr...	493	St@1	PORT			2
test.risc1.pgmctr...	492	St@0	PORT			1
muxout			PORT			2
in1			PORT			1
in0			PORT			2
test.risc1.pgm...	489	St@1	PORT			1
test.risc1.pgm...	664(Prime)	St@1	PORT			1
test.risc1.pgm...	488	St@0	PORT			1

For more details on the **FaultDetail** Window, see [Using the FaultDetail Window](#).

Support for GM+mFM FSDB File in Verdi

Verdi supports GM+mFM FSDB waveform file and also allows you to select the Fault Model. For more details, see the following subsections:

- [Selecting Fault ID From nTrace Window](#)
- [Displaying Fault ID of Secondary FSDB File in nTrace Window Title](#)
- [Selecting Fault ID From nWave Window](#)
- [Opening Same FSDB File in Different nWave Window](#)
- [Save Restore Session](#)
- [Changing Fault ID](#)
- [Reopening FSDB File in nWave](#)
- [Adding Fault Waveform](#)
- [Limitations](#)

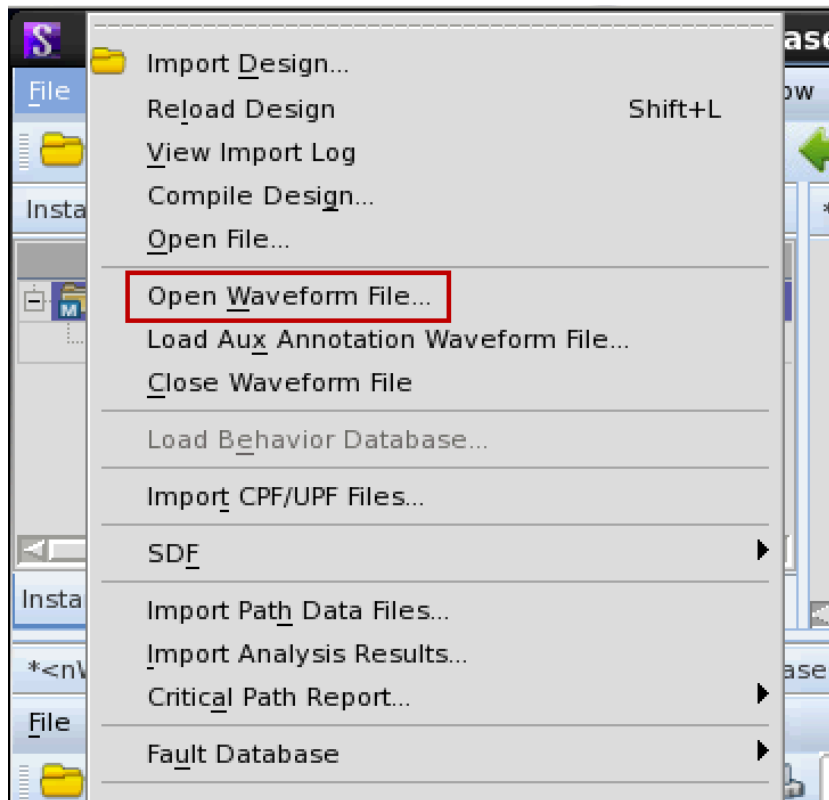
Selecting Fault ID From nTrace Window

You can select **Fault ID** when you open a waveform file from the *nTrace* window.

To select the **Fault ID** when you open a waveform file from the *nTrace* window, perform the following steps:

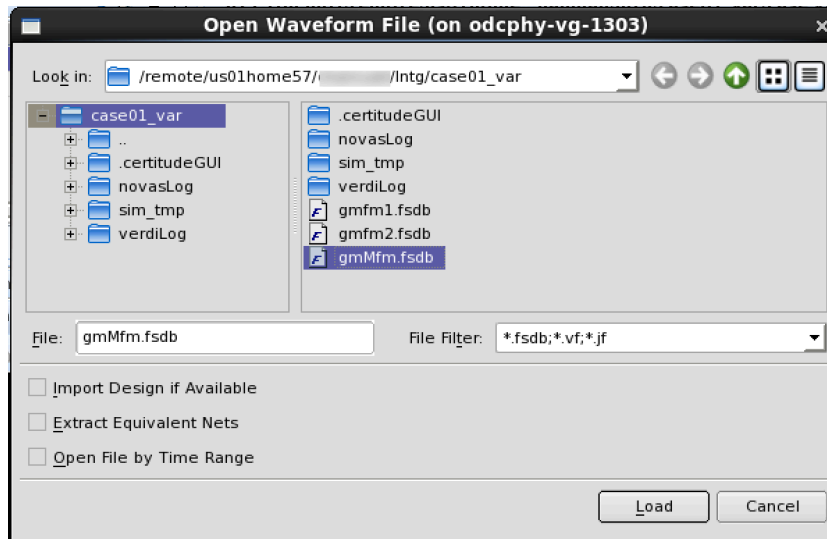
1. In the *nTrace* window, click **File > Open Waveform File**. This command displays the **Open Waveform File** dialog box.

Figure 1 *nTrace* Menu



2. In the **Open Waveform File** dialog box, select one FSDB file in **Open Waveform File** dialog box and click **Load**.

Figure 2 Open Waveform File



3. If this FSDB has multiple fault ID, the **Select Fault ID** dialog box is displayed.
4. Click the **Fault ID** drop-down box to select the fault ID and click **OK**.

Figure 3 Select Fault ID

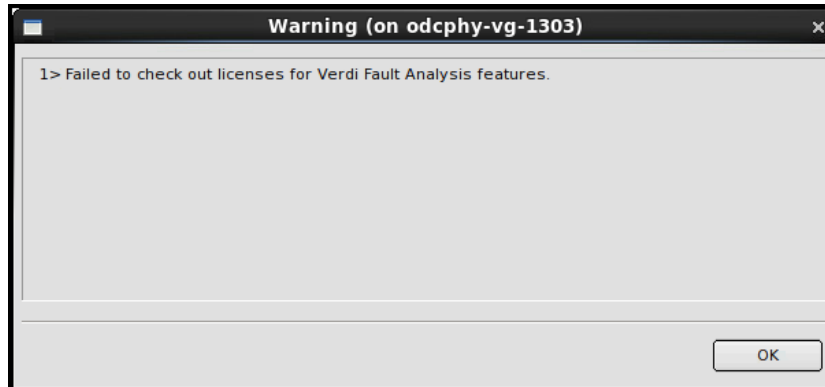


Note:

If the FSDB file has only one fault ID, the **Select Fault ID** dialog box does not appear. The only one fault ID is set automatically.

5. If the Verdi check license fails, a warning message appears as shown in the following figure:

Figure 4 Verdi License Warning

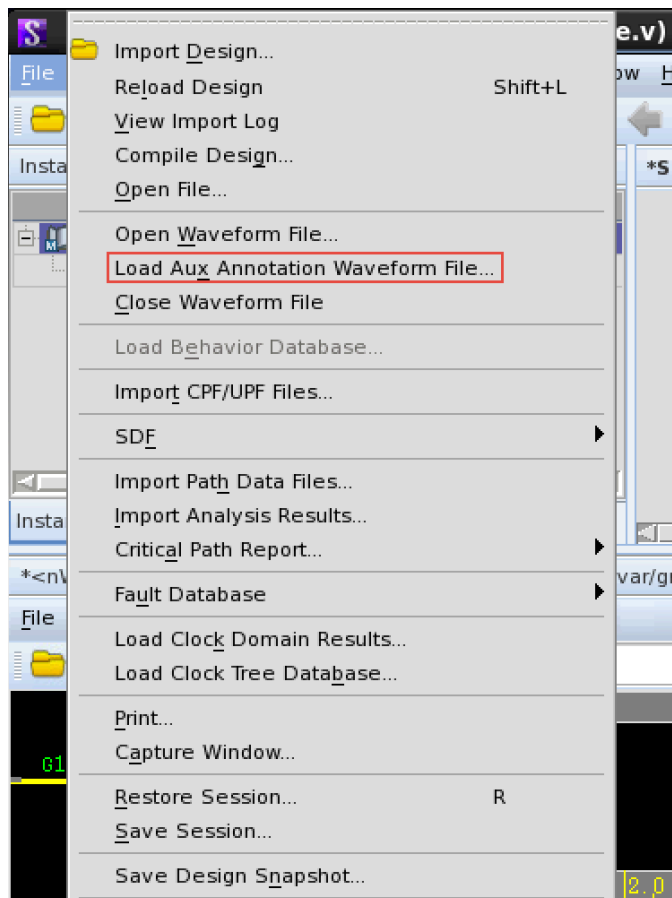


Displaying Fault ID of Secondary FSDB File in nTrace Window Title

If the secondary FSDB file has a fault ID, then the **Fault ID** is displayed in the *nTrace* window. For example, perform the following steps:

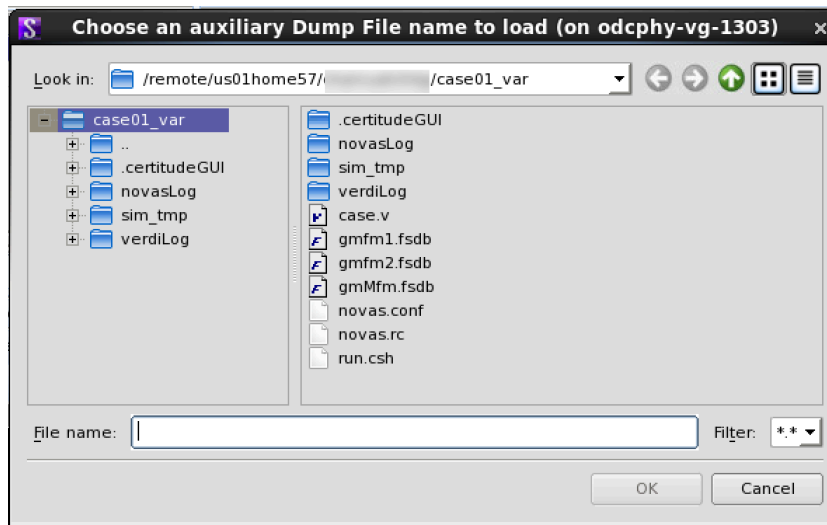
1. In the *nTrace* window, click **File > Load Aux Annotation Waveform File**. This command displays the **Choose an auxiliary Dump File name to load** dialog box.

Figure 5 *nTrace Menu Options*

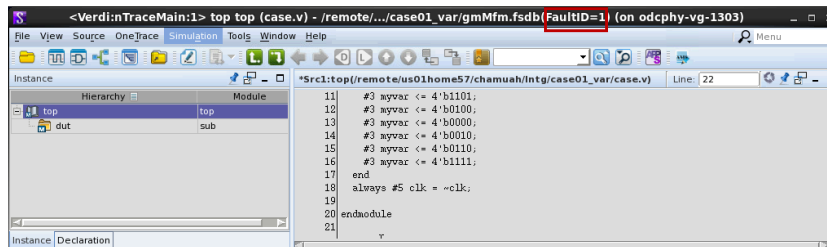


2. In the **Choose an auxiliary Dump File name to load** dialog box, select one FSDB file and click **OK**.

Figure 6 Choose an Auxiliary Dump File name to load



3. If the FSDB file has a fault ID, it appears on the *nTrace* window title.



Note:

The **Select Fault ID** dialog box is not displayed for Secondary FSDB file from *nTrace*.

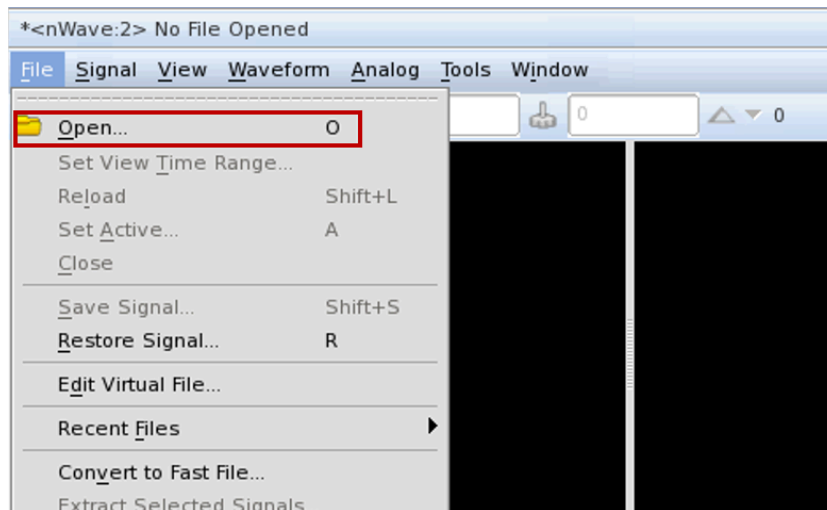
Selecting Fault ID From nWave Window

You can select **Fault ID** when you open waveform file from *nWave* window.

To select the **Fault ID** when you open waveform file from *nWave* window, perform the following steps:

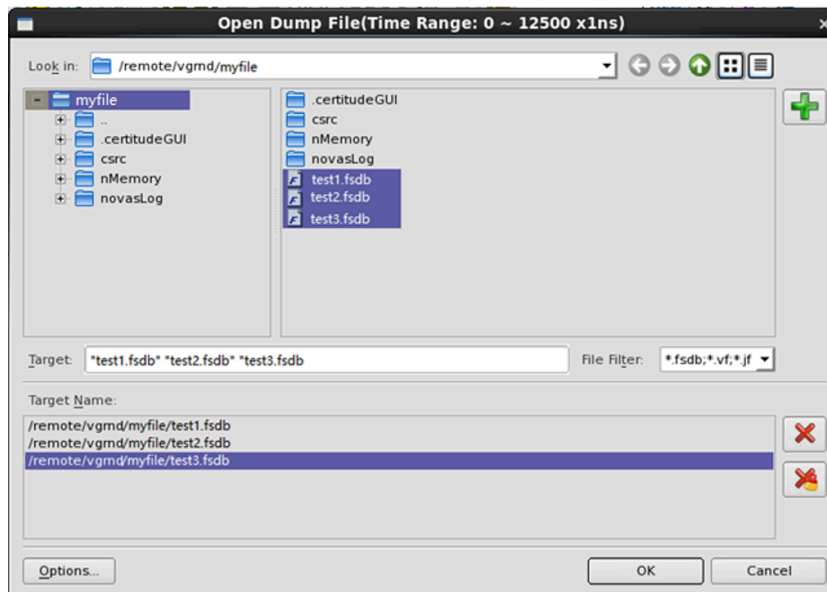
1. In the *nWave* window, click **File > Open**. This command displays the **Open Dump File** dialog box.

Figure 7 *nWave Menu*



2. Select one or more FSDB file, add to file list, and click **OK**.

Figure 8 *Open Dump File*



3. If the selected FSDB files contains a FSDB with multiple fault ID, then it displays the **Select Fault ID** dialog box and allows you to select the fault ID.

Figure 9 Multiple File in Select Fault ID



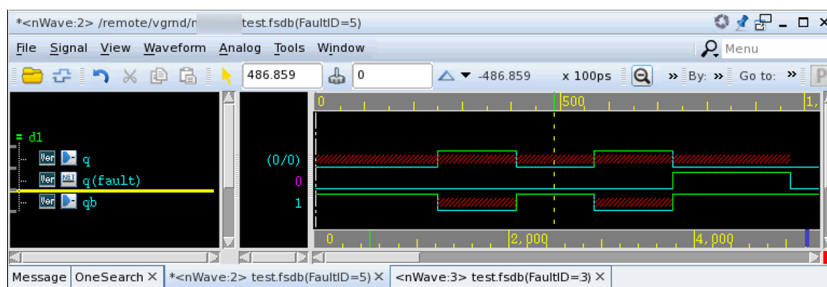
Opening Same FSDB File in Different nWave Window

When you open the same FSDB file in multiple *nWave* window, you have the option of selecting different fault ID in different *nWave*.

For example,

open `test.fsdb + fault 5` in **nWave2**, and open `test.fsdb + fault 3` in **nWave3**.

Figure 10 Open same FSDB file in different nWave



Save Restore Session

To save and restore session, add the fault ID keyword to save the selected fault ID in the `*.ses` file and the `*.ses.wave.*` file.

```
25 [fusa]
26 fdb_server =
27 fdb_proj =
28 fdb_campaign =
29 fdb_tool =
30 [hb]
31 postSimFile = /remote/vgrndn/myfile/test.fsdb
32 faultID = 5
33 syncTime = 486.859
```

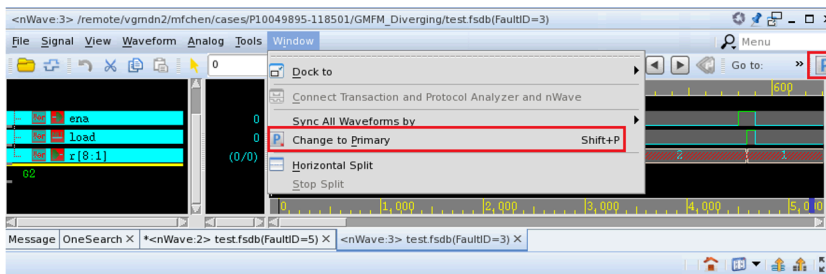
```
7 ; File list:
8 ; openDirFile [-d delimiter] [-s time offset] [-rf auto_bus_rule_file] path_name file_name
9 openDirFile -d / -faultID 5 "" "/remote/vgrnd/myfile/test.fsdb"
10
```

Changing Fault ID

You cannot change the fault ID in runtime once the FSDB file is opened. However, you can load the FSDB in different *nWave* window(s) with different fault ID(s) and then change the window to primary.

To change the window to Primary, click the **Window > Change to Primary** option.

Figure 11 Change to Primary

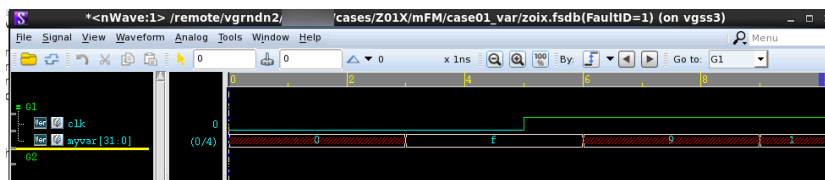


Reopening FSDB File in nWave

When you reopen GM+mFM FSDB in the *nWave* window, you can select the fault ID again. For example,

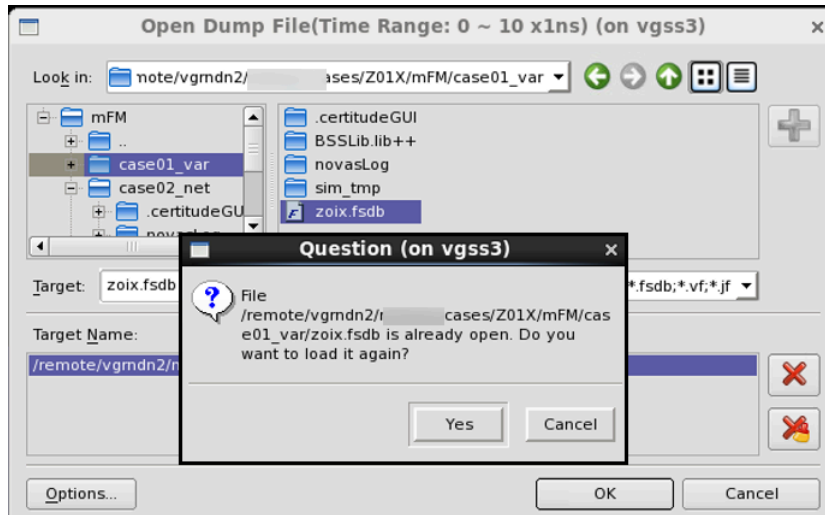
1. Open `z0ix.fsdb` file in **nWave1** and select fault ID 1.
2. Add signal `myvar[31:0]` to waveform, signal `myvar[31:0]` shows the fault ID 1 value.

Figure 12 Open file in nWave1, Select fault ID 1



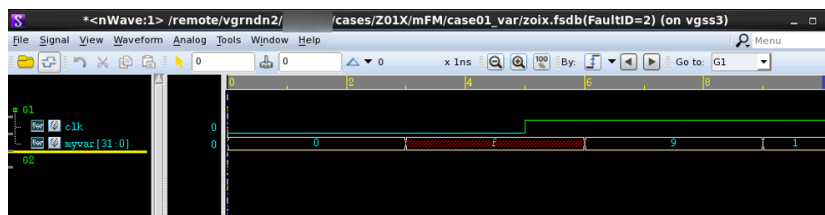
3. Re-open `zoix.fsdb` in *nWave* and select the fault ID 2.

Figure 13 Re-open file in *nWave*1



4. Signal `myvar[31:0]` shows fault ID 2 value.

Figure 14 Select fault ID 2, update waveform



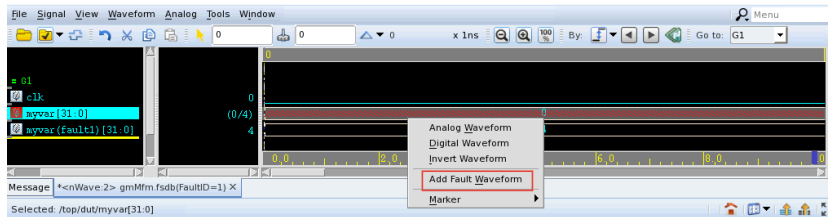
Note:

Reloading the design/Reload waveform file does not change the fault ID which has been set.

Adding Fault Waveform

You can add fault ID as postfix of fault waveform signal name by clicking the **Add Fault Waveform** in the RMB.

Figure 15 Add Fault Waveform



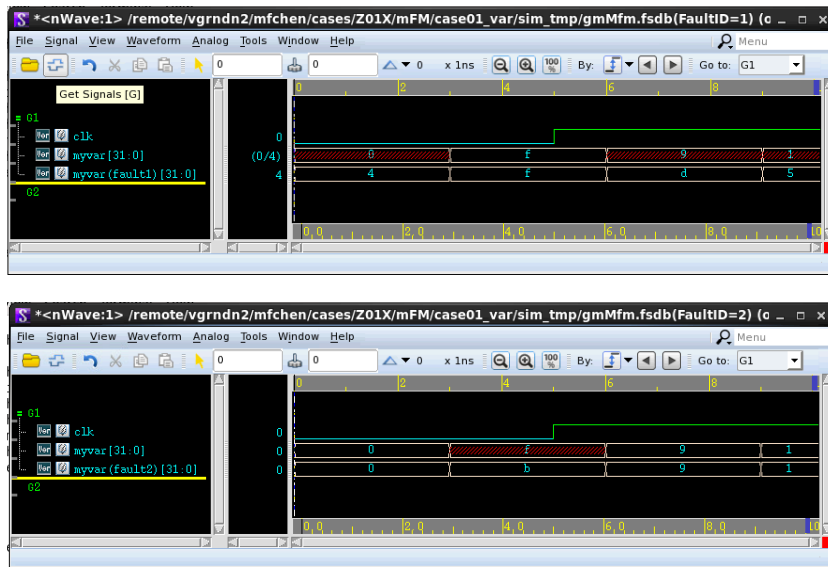
The fault waveform signal name is of the following format:

`gm_signal_name+(fault[fault_id])`

For example, when you open FSDB file `test1.fsdb` in **nWave1** and select faultID 2, the gm signal name is `myvar[31:0]`, the fault waveform signal name is `myvar(fault2)[31:0]`.

When you open FSDB file `test1.fsdb` in **nWave2** and select faultID 1, the gm signal name is `myvar[31:0]`, the fault waveform signal name is `myvar(fault1)[31:0]`.

Figure 16 Examples of Add Fault Waveform



Limitations

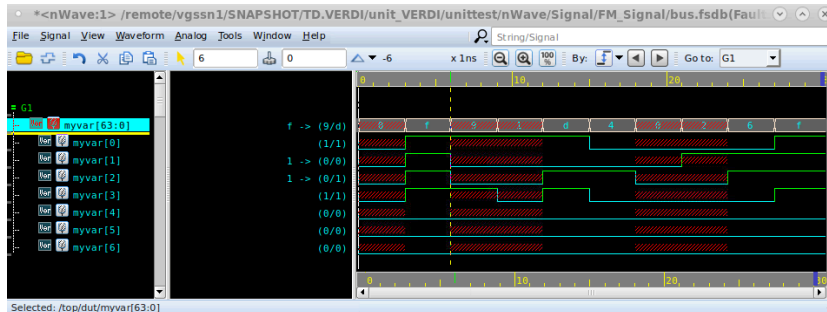
The following are the limitations of this feature:

- GM+mFM FSDB does not support *nWave* non-blocking drawing (`setenv VERDI_MULTITASKING_WAVEFORM`).
- Composite signal cannot show diverge value on parent signal.

- You can expand to see diverge value on members.
- Composite signal cannot create fault waveform.
- Bus signal cannot create fault waveform for its individual members.
- Virtual file is not supported in this project.
 - Switch type: You cannot open files that include GM/FM information.
 - Split/Stitch/SLVF type: The source FSDB file that includes GM/FM information is skipped while opening.
- FSDB utilities
 - `fsdbextract/fsdbedit`: The fault machine information is lost after extracting Z01X FSDB.
 - `fsdbreport`: It does not report the value of fault signals for Z01X FSDB.
 - `fsdb2saif`: It skips the fault signals for Z01X FSDB.
 - `fsdb2vcd`: It skips the fault signals for Z01X FSDB.
 - `fsdb2ns`: It skips the fault signals for Z01X FSDB.
- The color change feature for mismatch GM/FM is not supported in the *nEditSchematic* window.
- A mismatched annotation is limited to the fault signal or location where it is dumped.
 - Bus signals share the same mismatched annotation event.
 - You must trace all alias nets to have complete visibility.

nWave Display for GM/FM Waveform

nWave shows Golden Model (GM) waveform and marks differences between GM and Fault Model (FM) waveforms in diverge/converge range with red slash as shown in the following figure:



The value pane format is (GM_value/FM_value).

Composite signals does not show diverge value on parent signal. You can expand the waveform pane to see the diverge value on members.

For example, Packed Struct and Unpacked Struct members show diverge values as follows:

Packed Struct:

```
struct GW = { 00, 01 }; FW = { 00, 00 }  
- struct.f1 = (00/00) // since packed var is single idcode and it will  
  show (00/00)  
- struct.f2 = (01/00)
```

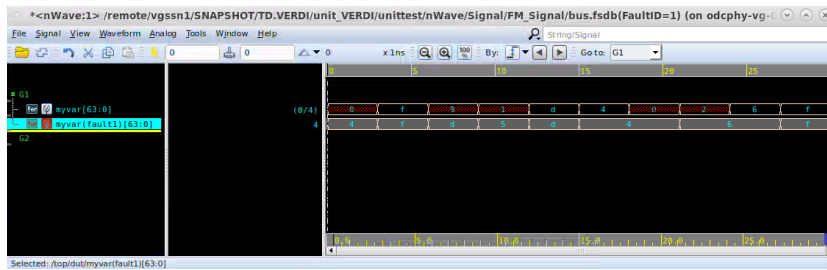
Unpacked Struct:

```
struct GW = { 00, 01 }; FW = { --, 00 }  
- struct.f1 = 00 // since unpacked var are multiple idcode and it only  
  show 00  
- struct.f2 = (01/00)
```

You can reuse current Mismatch to search diverge/converge ranges. In case of same time point, you can click the **Up** or **Down** icon to find the previous or next signal. You can click the **Next** or **Previous** icon to find the next/previous diverge range.

All other debug features such as logical operation, report, search by value, and coverage are applied only on GM waveform.

To add fault waveform on the signal, click RMB option **Add Fault Waveform**. This option adds a runtime signal under the GM signal to draw the fault waveform as shown in the following figure. The name of this runtime signal is "GM_signal_name" + "(fault)".



Display of Hold Time Pair for the Fault

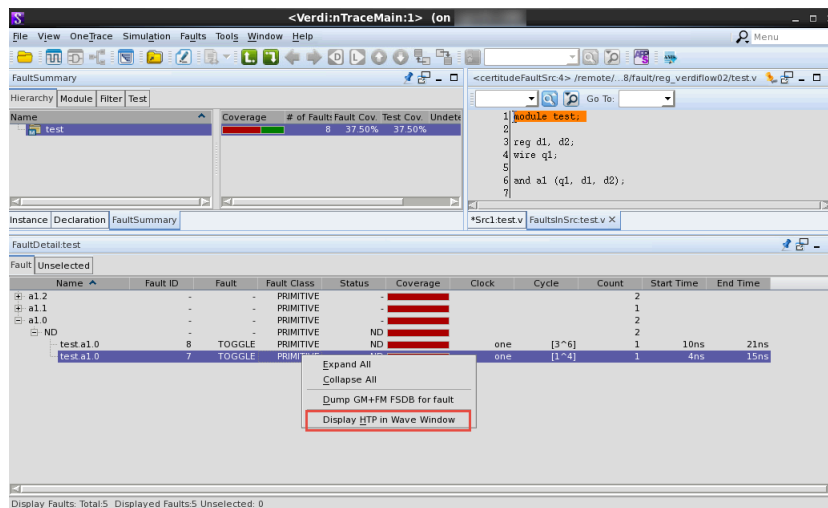
In Verdi-Coverage, you can use the keyboard shortcut key **Shift+Ctrl** to select multiple faults and click the **Update hold-time-pairs** button on the detail fault GUI to see the hold time of faults.

For the display of Hold Time Pair (HTP) for the fault, the following points should be considered:

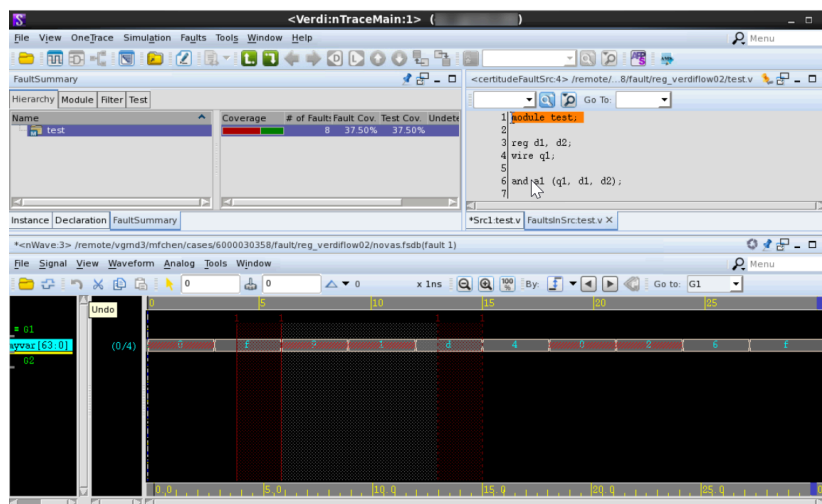
- Open the *nWave* window and load the FSDB file.
- You can open multiple *nWave* windows but only the Primary *nWave* window shows HTP. The primary waveform is annotated with a red square on the lower right corner of the waveform window. By default, the first *nWave* window to be invoked is the primary window.
- You can open multiple FSDB files in same *nWave* window but only one file is Active. To view Active file, go to **nWave > File > Active File**. The HTP displayed in primary *nWave* window is cleared, if you change the Active file.

Following are the steps to display HTP in the *nWave* window:

1. Double-click a module in the **FaultSummary** frame to open the **FaultDetail** View.
2. Open the *nWave* window and add signal, for example, `cmp[31:0]`.
3. Select a fault and click the RMB option **Display HTP in nWave Window**.



4. You can view the HTP in the *nWave* window.



5. Begin-end time range on the waveform can be seen in the waveform as follows:

- Solid line denotes start time (inject time) and dashed line denotes end time (release time). The line uses the fault-status color. Long dash orange line means more than one event (inject or release) occurring at the same time.
- Number labels denote lines (Hold Time Pair) that can be overlaid together.
- Shade color is used for background, fault-status color for non-overlap range, gray color for the overlapped range.

Comparing FSDB files Containing GM+nFM Data in nCompare

The ability to compare FSDB files will be imperative in being able to validate the data being generated by the dumper/FSDB file. Verdi supports comparing the FSDB data generated through new methods with FSDB files generated by existing features, and also to be able to detect potential differences when code is updated or fixed.

nCompare provides the following features:

- Command line option to specific faultId which needs to be compared

- **Syntax**

```
nCompare [-wave waveform_file1 [-wave waveform_file2] [-faultId  
fault_id1 [-faultId fault_id2]]
```

- **Example**

```
nCompare -wave gmfm1.fsdb -wave gmfm2.fsdb -faultId 3 -faultId 1  
nCompare -wave gmfm.fsdb -wave gmMfm.fsdb -faultId 2
```

- **Error Message**

Error message is printed and comparison is stopped if the faultId does not exist.

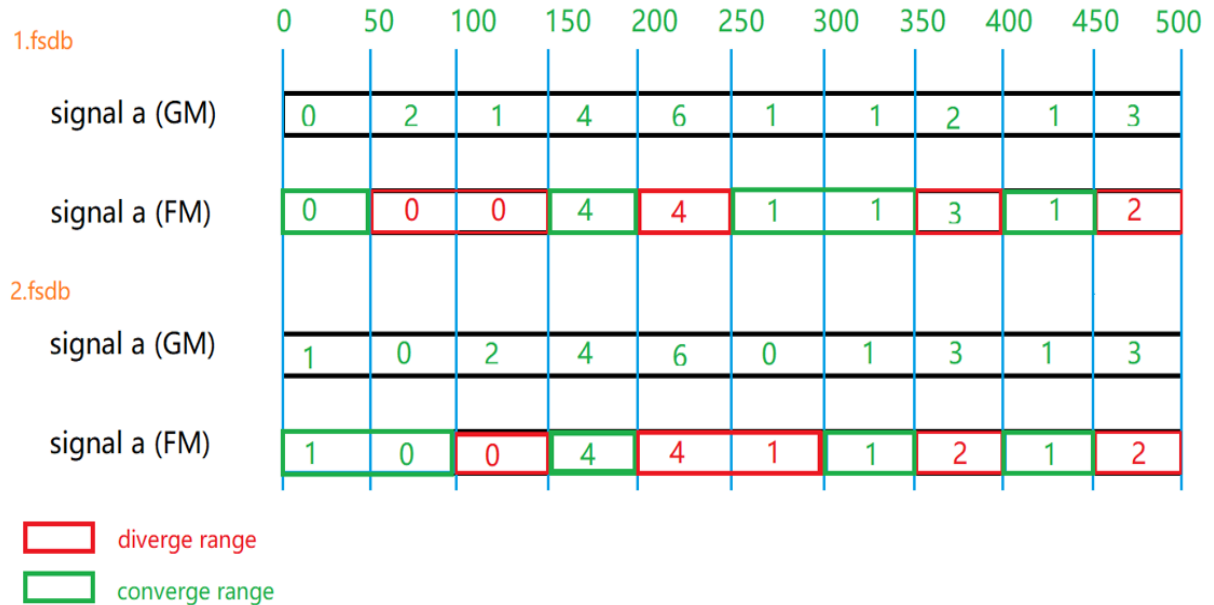
```
Command Line: File %s does not contain fault %d. Stop!
```

- Report differences in state (divergence/convergence) and value changes (in diverge range).

Algorithm

- If both golden and secondary FSDB have FM
 - if golden signal has no FM or secondary signal has no FM
 - skip
 - if both golden and secondary signal have FM
 - compare the FM state and value of the two signals at the same time:
 - If state (diverge/converge) is different, report mismatch
 - If state is the same, compare the FM value
- If golden FSDB has no FM or secondary FSDB has no FM
 - if golden signal or secondary signal has FM
 - compare the FM with other signal

- if both golden signal and secondary signal have no FM
 - compare the two signals normally



Mismatch:

- 50ns (1.a diverge; 2.a converge)
- 250ns (1.a converge; 2.a diverge)
- 350ns (1.a diverge, value 3; 2.a diverge, value 2)

Error File

- Reports faultId for a file.

Example

```
500
"/remote/vgssn1/SHOT/VERDI/unit_VERDI/unittest/nCompare/FMCompare/gmfm1.fldb"
"/remote/vgssn1/SHOT/VERDI/unit_VERDI/unittest/nCompare/FMCompare/gmfm2.fldb"
1n 1n fault1 fault2 2 1 1
```

- Report differences in state (divergence/convergence) and value changes.

Example

```
510 [G:start] [S:start] [G:width] [S:width] [G:signal] [S:signal]
    [common option] [G:option] [S:option] [G:value] [S:value] [Diff:value
    percentage]

510 1ns 1ns 15ns 15ns `/hw_top/out[19:1]` `/hw_top/out[19:1]` 3 1
    2 Diverge (GM:1101000000000000100, FM :1001000000000000100) Converge
    (GM:1101000000000000100, FM:1101000000000000100)

510 1ns 1ns 15ns 15ns `/hw_top/out[19:1]` `/hw_top/out[19:1]`
    3 1 2 Diverge (GM:1101000000000000110, FM:1101000000000000100)
    Diverge (GM:1101000000000000110, FM:1100000000000000100)
```

Compare Log

- Report the number of mismatches in each pair of signals.

Example

```
/hw_top/i_dut/i_types/a1[0:7] (fault %d) vs. /hw_top/i_dut/i_types/
a1[0:7] (fault %d) : %d errors.
```

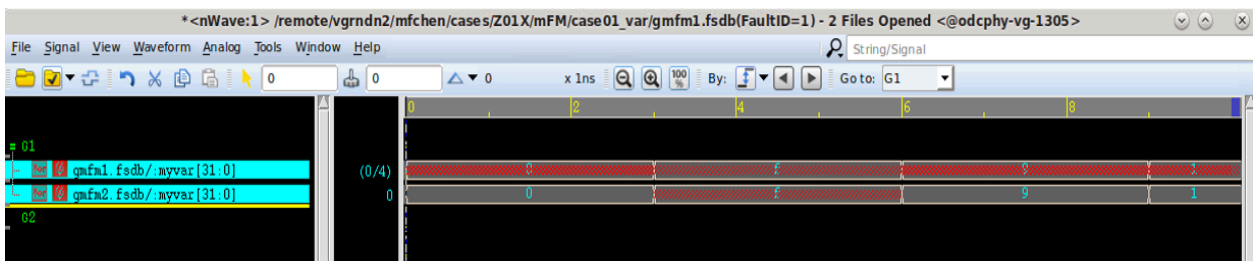
- Summary FM comparison information

There are %d FM pairs compared, %d FM pairs having errors and %d errors found.
There are %d pairs having no FM in golden signal, %d pairs having no FM in
secondary signal, %d pairs having no FM in both signals.

Displaying Error in nWave

The pattern of diverge range and mismatch range are the same, which results in the inability to distinguish the diverge range and mismatch range. So the diverge range pattern is changed to "SLANT_RIGHT" when signal need to show mismatch.

- Second signal:
 - FM pattern: ID_RED5, SLANT_RIGHT
 - Mismatch pattern: ID_RED5, SLANT_LEFT
- Golden signal:
 - FM pattern: ID_RED5, SLANT_LEFT



Limitation

When the golden FSDB file and the second FSDB file are the same, only the second FM value can be displayed in nWave.

